CS3 RESEARCH EXPERIENCE FOR TEACHERS

Introduction to Language Models

Autocompletion, tokens, embeddings, and sequence models

Mehmet Kerem Turkcan

Center for Smart Streetscapes, Columbia University

A language model is intelligent autocompletion

Fundamental idea

Intelligent autocompletion of a sequence, for example, a text message.

The keyboard reads what you have typed so far and suggests the words most likely to come next.

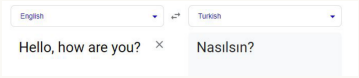


Autocompletion can pause for a change

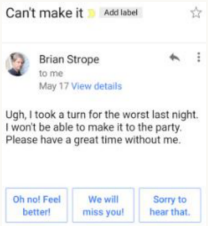
- Intelligent autocompletion of a sequence, for example, a text message.
- The “autocompletion” might have a “pause” in between, corresponding to a “change”: from one language to another, or from one speaker to the next.



Predictive text



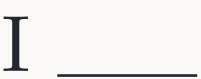
Translation



Suggested replies

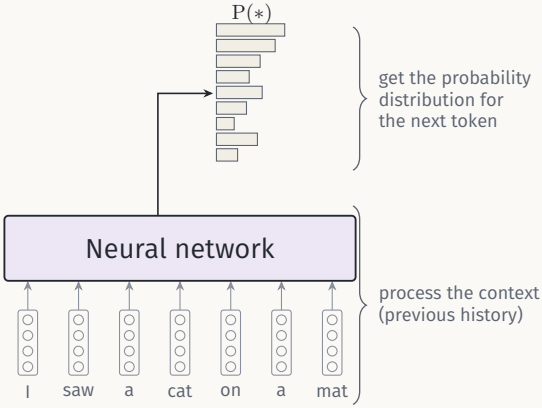
Tokens, and the question the network answers

- Split a sequence into **tokens** using a **tokenizer**.
  - Text: words, single letters, syllables, or *made-up syllables that minimize the total number of "syllables" in a collection of documents*.
  - Images: "patches" of an image.
  - Videos: individual video frames.
  - Audio: 1-second sections of the audio.
- Use a neural network to answer one question: given a series of tokens, what is the probability of each token being the next one?



What comes next?

From context to a next-token distribution

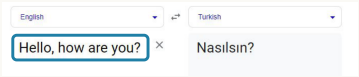


Special tokens and context

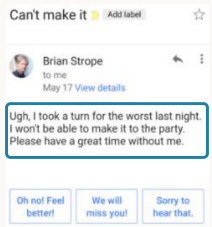
- For translation and chatting, add a **special token** that marks where the change happens.
- **Context**: what the model sees, which determines how far back it can observe. Outlined below.



Predictive text



Translation

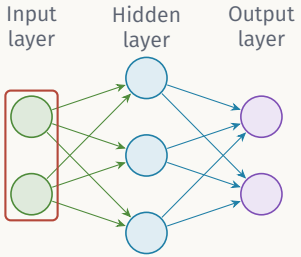


Suggested replies

How do we calculate the inputs from tokens?

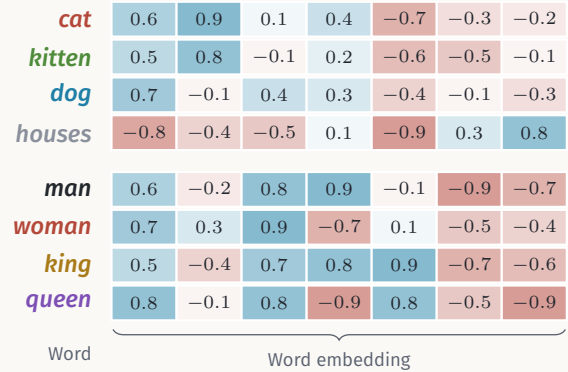
A network's input layer takes numbers, but a token is a symbol: a word, a syllable, or a patch.

Which numbers should go into the highlighted layer?



Embeddings: a dictionary from tokens to vectors

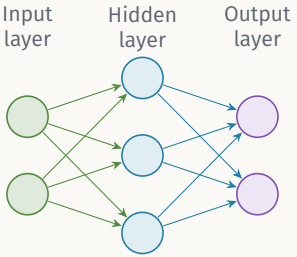
- Each token corresponds to a collection of numbers, a vector, that serves as the input layer of the model.
- We update these vectors during training as if they were another hidden layer. We call them **embeddings**.
- Think of a dictionary in which we look up a hidden layer for each token, instead of using an input given to us.
- To **decode** a vector into a token, find the closest token in the dictionary.



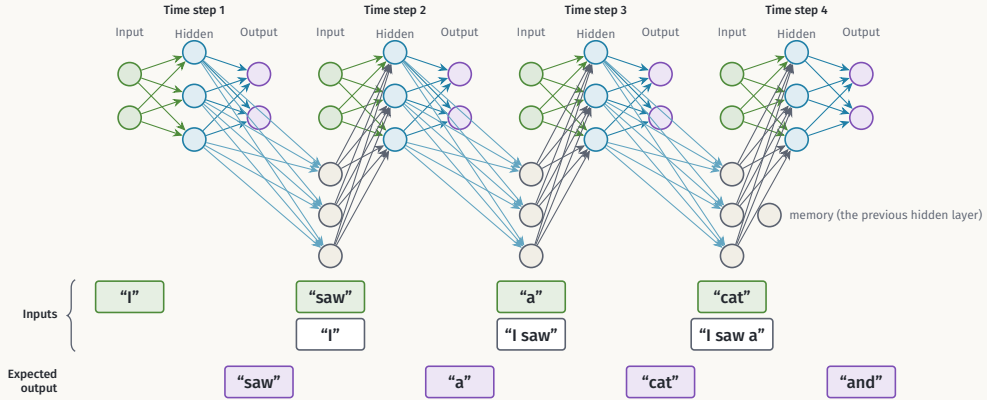
Cell shading: blue for positive values, red for negative values, darker for larger magnitudes.

A feedforward network maps one input to one output

- Information flows in one direction: input layer, hidden layer, output layer.
- The network has no memory of earlier inputs.



A recurrent network carries memory forward



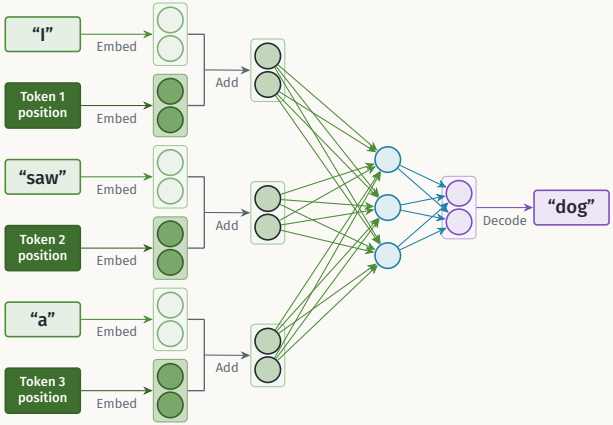
Green: the current token. Gray: what the memory has accumulated. Purple: the token the network should predict.

Convert time to vectors too

Fundamental idea

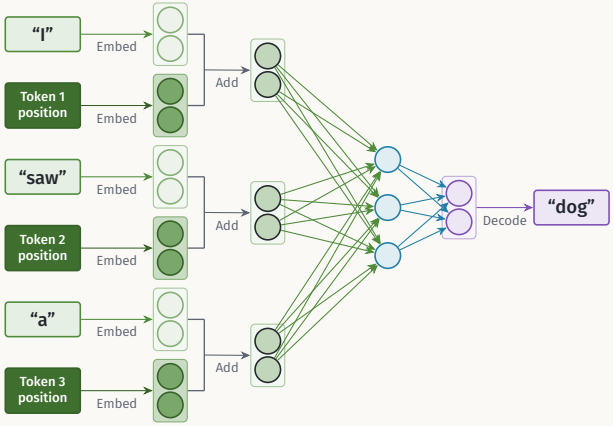
We have converted tokens to vectors, so why not convert time to vectors as well?

- Then we can forget about causality.
- Large language models follow this idea.

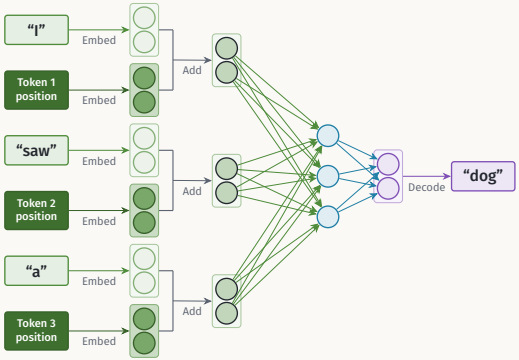


Add position embeddings to token embeddings

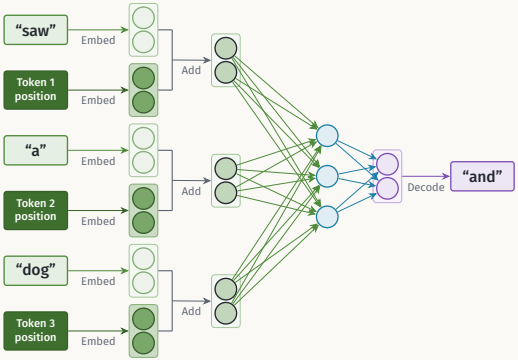
1. Keep a dictionary that maps time steps to vectors: **positional embeddings**.
2. Embed the tokens.
3. Embed the positions.
4. Add the token vectors to the position vectors.
5. Train a neural network to predict the next word.
6. Shift the input tokens by 1, adding the prediction as if it were part of the starting context.



Shift by one token and predict again



Step 1: “I saw a” predicts “dog”.



Step 2: the prediction joins the context; “saw a dog” predicts “and”.

Images can also be modeled this way

- Cut the image into patches and read them in order, like words in a sentence.
- The model predicts the next patch from the patches it has seen.

